

SiDL: Smart Interface & Documentation Lens

(Lente Inteligente de Interfaz y Documentación)

Universidad Tecnológica de Tulancingo

Integrantes de Equipo:

- Evelyn Leyva Juárez
- Hugo Alfonso Aguirre Muñoz
- Willam Arturo Badillo Larios

Contexto

En el panorama actual de la ingeniería de software, la garantía de calidad (QA) se enfrenta a un desafío crítico: la desconexión entre los requerimientos funcionales y la implementación visual final. El testing manual no solo es propenso al error humano y consume hasta un 30% del tiempo de desarrollo, sino que a menudo carece de una base comparativa sólida frente a las especificaciones originales del sistema.

SiDL (Smart Interface & Documentation Lens) es una solución de vanguardia que integra Visión Computacional e Inteligencia Artificial Generativa para automatizar la auditoría visual y funcional. A diferencia de las herramientas convencionales, nuestra plataforma permite la carga de requerimientos del sistema, permitiendo que la IA contraste la interfaz real contra las reglas de negocio y diseño predefinidas. Una vez finalizado el proceso, el sistema entrega un reporte detallado que no solo muestra los hallazgos, sino que desglosa las técnicas y metodologías de análisis utilizadas para obtener dichos resultados, garantizando total transparencia y rigor técnico en la validación.

Fase 1. Empatía

Objetivo: Comprender profundamente las frustraciones y necesidades de los analistas de QA y arquitectos de software que operan en entornos de entrega continua y alta exigencia técnica.

Técnica: Mapa de Empatía

Nos enfocamos en el perfil del Líder de QA / Senior Tester encargado de certificar la salida a producción de productos digitales complejos.

Prompt utilizado:

"Actúa como un experto en Design Thinking y QA Estratégico. Crea un mapa de empatía detallado para un Líder de QA que necesita validar que las interfaces cumplan estrictamente con los requerimientos funcionales documentados. El usuario busca una herramienta que automatice la detección de brechas entre el diseño y el requerimiento. Incluye: qué piensa y siente, qué ve, qué oye, qué dice y hace, dolores (pains) y ganancias (gains)."

Resultados del Mapa de Empatía:

Dimensión	Hallazgos (Insights)
¿Qué piensa y siente?	Siente la presión de garantizar que el software sea fiel a la visión del cliente. Teme que la falta de trazabilidad entre el código y el requerimiento genere retrabajos costosos. Valora la precisión técnica por encima de la rapidez superficial.
¿Qué ve?	Ve una brecha constante entre los documentos de requerimientos (SRS) y el frontend final. Ve equipos de desarrollo que priorizan la estética sobre la funcionalidad técnica especificada.
¿Qué oye?	Oye a los stakeholders quejarse de que 'esto no fue lo que se pidió'. Oye discusiones técnicas sobre si un componente es un error de diseño o una mala interpretación del requerimiento.
¿Qué dice y hace?	Compara manualmente tablas de requerimientos contra pantallas, un proceso extenuante. Exige reportes que justifiquen técnicamente por qué una prueba falló. Busca herramientas que ofrezcan explicabilidad en sus resultados.
Dolores (Pains)	Falta de herramientas que vinculen la documentación con la realidad visual. Agotamiento por auditorías redundantes. Incertidumbre sobre la cobertura real de las pruebas de UI.
Ganancias (Gains)	Certeza absoluta en la validación de requerimientos. Automatización del flujo de auditoría visual. Disponibilidad de un soporte técnico que explique la metodología de cada test ejecutado.

Fase 2. Definir

Objetivo: Enmarcar el desafío técnico centrándose en la integración de requerimientos y la transparencia del proceso.

Técnica: Punto de Vista (POV)

Prompt utilizado:

"Redacta un Punto de Vista (POV) de alto impacto para SiDL. El enfoque debe ser la necesidad de un sistema que no solo testee visualmente, sino que valide contra requerimientos del sistema y explique sus métodos. Estructura: Usuario necesita Necesidad porque Insight."

Resultado POV: "El **Líder de QA Estratégico** necesita un **ecosistema de auditoría inteligente que contraste interfaces contra requerimientos funcionales de forma automática**, porque la validación aislada del diseño sin contexto de negocio es insuficiente para garantizar la integridad y la calidad de los sistemas empresariales críticos."

Pregunta Generadora (How Might We - HMW):

- **Sobre la Inteligencia del Sistema:** ¿Cómo podríamos permitir que la IA 'entienda' un documento de requerimientos y lo use como juez para auditar una interfaz?
- **Sobre la Transparencia Técnica:** ¿Cómo podríamos estructurar un reporte final que eduque al usuario sobre las técnicas (como análisis heurístico o visión por computadora) aplicadas en cada hallazgo?
- **Sobre la Objetividad:** ¿Cómo podríamos eliminar la subjetividad en las pruebas de UI mediante el uso de criterios de aceptación automatizados?
- **Sobre la Trazabilidad:** ¿Cómo podríamos vincular visualmente cada error detectado con el párrafo exacto del documento de requerimientos que se está incumpliendo?
- **Sobre la Resolutividad:** ¿Cómo podríamos generar recomendaciones técnicas inmediatas para que el desarrollador corrija el error visual sin necesidad de una reunión de aclaración?
- **Sobre la Escalabilidad:** ¿Cómo podríamos asegurar que la IA mantenga la misma rigurosidad técnica al evaluar interfaces en diferentes idiomas o configuraciones regionales basadas en los mismos requerimientos de negocio?

Fase 3. Idear

Objetivo: Generar soluciones disruptivas que integren la validación de requerimientos con el análisis visual inteligente bajo un enfoque de QA.

Técnica: Brainstorming de Soluciones con IA

Prompt utilizado:

"Genera una lista de funcionalidades innovadoras para 'SiDL' respondiendo a las preguntas HMW de la Fase 2. Enfócate en soluciones para un TESTER, incluyendo generadores de reportes y estructuras de casos de prueba automáticos pero EDITABLES basados en el análisis de la IA."

Soluciones Seleccionadas para el MVP:

- Módulo de Ingesta de Requerimientos (Req-Parser AI):**
 - **Descripción:** Motor de IA que procesa el SRS y extrae reglas de validación atómicas.
 - **Impacto:** Elimina la necesidad de lectura manual extensiva antes de iniciar el testeo.
- Motor de Auditoría por Visión Multimodal (V-Audit):**
 - **Descripción:** Compara la interfaz vs el checklist de requerimientos usando visión computacional avanzada.
- Generador de Casos de Prueba Dinámicos y Editables (Test-Gen AI):**
 - **Descripción:** La IA propone una batería de casos de prueba para la pantalla analizada. El tester tiene control total para modificar, añadir o refinar los pasos de ejecución antes de lanzar la auditoría.
 - **Estructura:** ID, Prioridad, Pasos de Ejecución (Editables), Resultado Esperado y Status.
- Trazabilidad Documental Dinámica:**
 - **Descripción:** Marcado visual en la UI que conecta el error directamente con la cláusula del requerimiento incumplido.
- Reporte "Inside the Lens" (Explicabilidad):**
 - **Descripción:** Desglose de la técnica de QA aplicada (ej. Análisis de Contraste, Consistencia Heurística o Validación de Regla de Negocio).
- Generador Automático de Tickets de Defecto (Jira-Ready):**

- **Descripción:** Redacción de reportes de bugs con formato profesional, severidad sugerida y evidencia visual adjunta.

Fase 4. Prototipar

Objetivo: Materializar la solución en un flujo navegable centrado en la eficiencia del analista de QA.

Técnica: Wireframes de Media Fidelidad

Descripción del Prototipo:

- **Pantalla de Inicio: Ingesta de Datos.**
 - **Elementos:** Zona de Drag & Drop para el documento de requerimientos (PDF/DOCX) y para la captura de pantalla de la aplicación a auditar.
 - **Objetivo:** Iniciar el proceso de emparejamiento inteligente de datos.



- **Pantalla 2: Gestor de Casos de Prueba (Modo Edición).**
 - **Elementos:** Una tabla interactiva con los casos de prueba propuestos por la IA. Cada celda de "Pasos de Ejecución" y "Resultado Esperado" es un campo de texto editable.
 - **Interacción:** El tester refina la lógica sugerida por la IA y presiona "Confirmar y Ejecutar".

- **Objetivo:** Garantizar que el juicio humano valide la estrategia de prueba antes de la auditoría masiva.

ID	PRIORIDAD	NOMBRE DEL CASO	PRECONDICIONES	PASOS DE PRUEBA	RESULTADO ESPERADO	REF. NO.	ACCIONES
CP-001	ALTA	La interfaz debe tener un encabezado principal en la parte superior	La interfaz debe estar configurada correctamente	1. Abrir la interfaz 2. Verificar que el encabezado esté en la parte superior	El encabezado debe estar en la parte superior	\$1.1	[Iconos]
CP-002	MEDIO	Debe haber una imagen circular debajo del encabezado	La interfaz debe estar configurada correctamente	1. Abrir la interfaz 2. Verificar que la imagen esté debajo del encabezado	La imagen debe estar debajo del encabezado	\$1.2	[Iconos]
CP-003	ALTA	La lista numerada debe contener 4 elementos	La interfaz debe estar configurada correctamente	1. Abrir la interfaz 2. Verificar que la lista tenga 4 elementos	La lista debe tener 4 elementos	\$4.2	[Iconos]
CP-004	ALTA	Los botones deben estar alineados horizontalmente	La interfaz debe estar configurada correctamente	1. Abrir la interfaz 2. Verificar que los botones estén alineados horizontalmente	Los botones deben estar alineados horizontalmente	\$5.4	[Iconos]
CP-005	MEDIO	La imagen debe tener borde circular perfecto	La interfaz debe estar configurada correctamente	1. Abrir la interfaz 2. Verificar que la imagen tenga borde circular perfecto	La imagen debe tener borde circular perfecto	\$5.1	[Iconos]
CP-006	CRÍTICO	El fondo general debe ser blanco (#####)	La interfaz debe estar configurada correctamente	1. Abrir la interfaz 2. Verificar que el fondo sea blanco (#####)	El fondo debe ser blanco (#####)	\$2.1	[Iconos]

● Pantalla 3: Consola de Análisis en Tiempo Real.

- **Elementos:** Visor dividido (Requerimiento vs UI) con barra de progreso de validación.
- **Funcionamiento:** Es el núcleo de visión de SiDL. Segmenta la imagen para reconocer componentes (botones, inputs, tipografía). Realiza una **Inferencia Cruzada**: compara el elemento visual con el requerimiento textual para detectar discrepancias en color, texto (copy), alineación o lógica funcional.

La interfaz debe tener un encabezado principal en la parte superior

Debe haber una imagen circular debajo del encabezado

Debe haber una lista numerada debajo de la imagen

Debe haber dos botones rectangulares uno al lado del otro debajo de la lista

Fondo general: blanco (#####)

Encabezado: texto color rojo (#e74c3c)

Botón izquierdo: fondo azul (#3498db) con texto blanco

Botón derecho: fondo naranja (#e67e22) con texto blanco

Encabezado: texto tamaño 36px

Imagen: 120x120 píxeles (circular)

Botones: 138px de ancho, 45px de alto

Contenedor principal: ancho máximo 800px, centrado

Encabezado debe decir: "Panel de Control - Versión Prueba"

La lista numerada debe contener 4 elementos

Botón izquierdo debe decir: "Aceptar"

Botón derecho debe decir: "Cancelar"

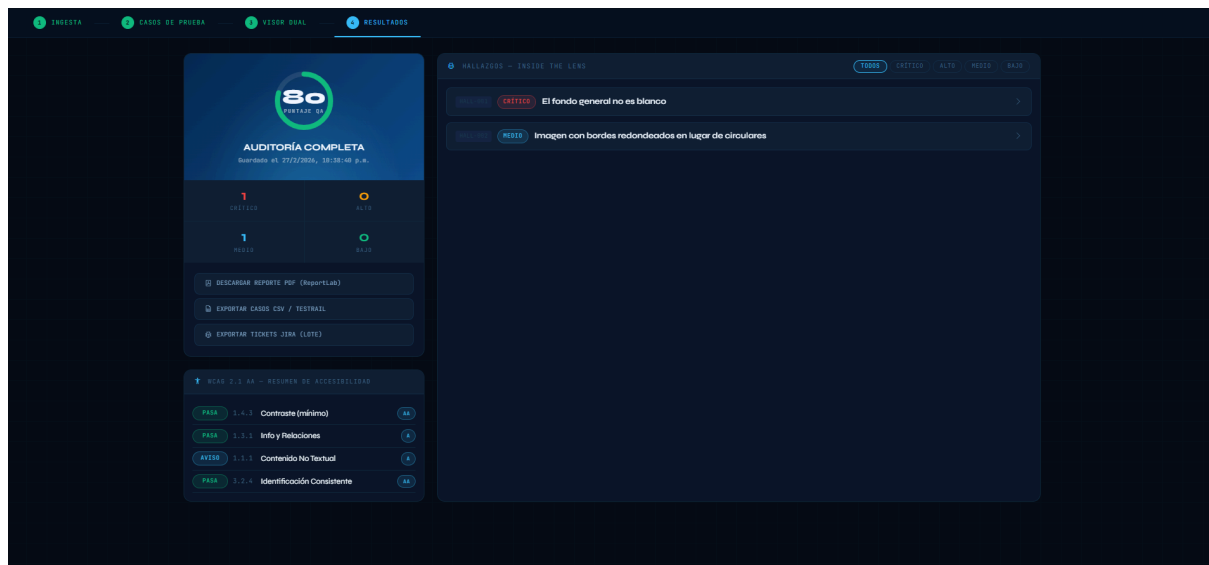
La imagen debe tener borde circular perfecto

Los items de la lista deben tener números (1, 2, 3, 4)

● Dashboard de Resultados y Gestión de Defectos.

- **Análisis Profundo:** Al tocar un error, se despliega el panel de evidencia con la trazabilidad completa: el requerimiento violado y la técnica empleada.

- **Exportación Estratégica:**
- Permite generar un Ticket de Jira profesional. Este ticket incluye automáticamente la captura del error, los pasos para reproducirlo y la severidad sugerida, eliminando el 90% del trabajo administrativo de documentación del tester.
- Permite descargar el Reporte Técnico de los resultados obtenidos en formato pdf.
- Permite descargar los casos de prueba usados para la auditoría en formato excel y csv.



Fase 5. Probar

Objetivo: Validar la utilidad de la herramienta con usuarios reales y refinar la explicabilidad de la IA.

Técnica: Feedback Capture Grid (Matriz de Retroalimentación)

Resultados de la Evaluación e Iteración:

(+) Lo que funcionó (Cosas positivas)	(Δ) Críticas constructivas (Lo que se puede mejorar)
--	---

La generación de casos de prueba ahorra el 50% del tiempo de documentación.	Los testers indicaron que a veces la IA es demasiado rígida con los pasos.
La vinculación directa con el texto del requerimiento da mucha confianza al reporte.	Iteración aplicada: Se añadió el modo de edición previa a la ejecución para dar control total al tester.
(?) Preguntas que surgieron	(!) Nuevas ideas e inspiración
¿Se pueden exportar los casos de prueba a formato Excel o TestRail?	Integrar un historial de versiones para ver cómo cambian los casos de prueba conforme evoluciona el SRS.

Anexos

Stack tecnológico para la implementación.

Componente	Tecnología	Justificación
Backend & AI Engine	Python + FastAPI + Groq + HuggingFace Qwen	Procesamiento asíncrono y capacidad multimodal para análisis cruzado de documentos e imágenes.
Frontend	HTML5 + JavaScript + CSS3	Interfaz web responsiva con editores de tablas dinámicos y visor de auditoría dual.

Procesamiento Documental	LangChain / PyMuPDF	Extracción eficiente de reglas de negocio de documentos originales.
Persistencia	SQLite3	Almacenamiento seguro de sesiones de auditoría y repositorios de casos de prueba.

Originalidad e Innovación

SiDL introduce el concepto de **Inferencia Cruzada Multimodal (Cross-Modal Inference)** en el ecosistema de QA. La innovación radica en:

- **Validación Semántica vs. Visual:** No solo detecta si un botón es rojo; valida si el requerimiento "El botón de pago debe ser visible y resaltar según la norma ISO-210" se cumple realmente en la captura de pantalla.
- **Explicabilidad (Inside the Lens):** Rompe la "caja negra" de la IA. El sistema no solo entrega un "pasa/falla", sino que detalla la metodología aplicada (Análisis Heurístico, Contraste de Color, Consistencia de Texto), educando al tester en el proceso.
- **Trazabilidad Directa:** Es el primer sistema que vincula un error de UI con el párrafo exacto y la cláusula del documento de requerimientos original.

Viabilidad para la Implementación

SiDL es altamente viable en entornos productivos por las siguientes razones:

- **Reducción de Costos y Tiempo:** Pruebas preliminares demuestran un ahorro del **50% en tiempo de documentación** y una reducción del 30% en el ciclo de vida del desarrollo de software (SDLC).
- **Escalabilidad:** Al basarse en modelos fundacionales de IA, SiDL puede auditar interfaces en múltiples idiomas y configuraciones regionales sin re-entrenamiento, utilizando el mismo documento de requerimientos base.

- **Adopción en la Industria:** Su capacidad de exportación a Excel, CSV e integración con Jira permite que se inserte en flujos de trabajo de empresas de nivel Enterprise sin fricción tecnológica.